TRAINOCATE

Module 2

Database performance and incident diagnostics

One day • 390 instructional minutes

Investigate → explain → mitigate → verify → hand over

TRAINOCATE

Today’s route

35 min | Database baseline and safe access
40 min | MySQL slow-query capture and ranking
35 min | EXPLAIN, indexes and misleading assumptions
40 min | MySQL blocking and deadlocks
15 min | Replication delay and reporting freshness
40 min | SQL Server requests, sessions and wait deltas
40 min | Blocking chains, Query Store and job failures
50 min | Application/database correlation
75 min | Final database incident challenge
20 min | Checkpoints and recovery reserve

The lab system and its boundaries

LEARNER

SSH + individual account

Team workspace

APPLICATION HOST

Tomcat + MySQL

Loyalty training app

DATABASE LAB

SQL Server 2022

Query Store + Agent

Request IDs connect application evidence.

Synthetic packs cover replication, IIS, SELinux and pooled-app incidents.

Fault injection and storage administration are instructor-controlled.

How to work an incident

1. State impact and capture UTC time.
2. Preserve evidence before changing the system.
3. Test a hypothesis with a discriminating check.
4. Make one scoped change with a rollback.
5. Verify the affected business function and communicate.

TRAINOCATE

Database baseline and safe access

The application reports database errors. Establish engine, endpoint, identity and connectivity before tuning anything.

- Use a protected client option file instead of putting passwords in command arguments.
- Read-only application rights differ from administrative diagnostic permissions.
- Capture UTC time, engine version and connection counts with every baseline.

Your investigation sequence

1. Create a protected client configuration
2. Enter the client settings
3. Run the bounded health check
4. Enter MySQL and identify the session
5. Capture a read-only baseline
6. Inspect service and full diagnostics

Create a protected client configuration

```
umask 077
touch "$HOME/.support-lab.cnf"
chmod 600 "$HOME/.support-lab.cnf"
nano "$HOME/.support-lab.cnf"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Baseline identifies endpoint and effective database user.
- ✓ Learner distinguishes access denied from server unreachable.

? Can a successful SELECT 1 prove every business query is fast?

? Does SHOW PROCESSLIST necessarily show all users to an unprivileged login?

Debrief and deliverable

A database baseline and a protected client configuration, never submitted with its password.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

MySQL slow-query capture and ranking

Player reports are slow. Capture an intentional slow query and rank work by both frequency and total time.

- Slow logging is a configuration change and must be restored after the exercise.
- Global long_query_time becomes the default for new sessions; existing sessions may retain their value.
- Frequency, total time, rows examined and lock time answer different questions.

Your investigation sequence

1. Record the original logging configuration
2. Enable a short capture window
3. Open a fresh learner connection
4. Create bounded slow-query evidence
5. Inspect the capture and rank the fixture
6. Restore the exact original values

Open a fresh learner connection

```
mysql --defaults-extra-file="$HOME/.support-lab.cnf"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Slow record appears with query time, lock time and row counts.

✓ Report explains why a frequent moderate query may cost more overall than one rare slow query.

? Does Query_time alone prove CPU pressure?

? Will changing a global threshold rewrite existing sessions?

Debrief and deliverable

A ranked slow-query table with units, frequency and next checks.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

EXPLAIN, indexes and misleading assumptions

The report scans many transactions. Decide whether a targeted index improves this workload.

- EXPLAIN estimates access strategy and cardinality; it is not proof of actual elapsed time.
- Indexes add write/storage costs and must be justified by workload.
- A tiny training dataset may show plan differences without dramatic timing differences.

Read the plan before proposing an index

Before: ALL / 100,000 rows → **After: ref / 1,000 rows** → **Verify: time + same result**

Illustrative EXPLAIN fields: type describes access; rows is an estimate. Measure actual work on the same input; an index also costs writes and space.

Connect and record the baseline

```
mysql --defaults-extra-file="$HOME/.support-lab.cnf"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Before/after plans and equivalent query results are retained.
- ✓ Recommendation includes an index tradeoff and measurement limitation.

? Is every full scan bad?

? Can an index be present yet unused?

? Does LIMIT guarantee cheap execution?

Debrief and deliverable

A before/after plan comparison and an evidence-based recommendation.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

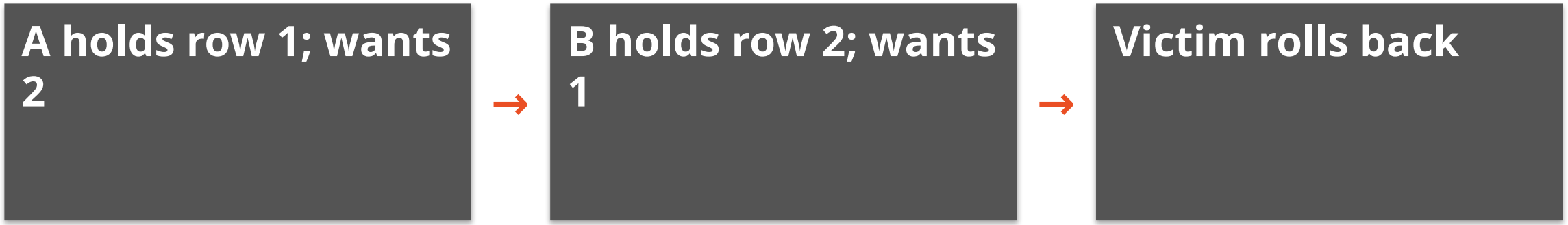
MySQL blocking and deadlocks

Two concurrent updates stall. Separate an ordinary lock wait from a deadlock and recover both sessions.

- A blocking chain can persist without a cycle; a deadlock requires a cycle.
- The deadlock victim is not necessarily the root cause.
- An open transaction can hold locks even while its client appears idle.

TRAINOCATE

A wait becomes a cycle



Ordinary blocking has a waiter; a deadlock has a cycle. Verify both sessions after release.

Open all three MySQL sessions

```
mysql --defaults-extra-file="$HOME/.support-lab.cnf" support_lab
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Evidence shows the blocking relation or deadlock cycle.
- ✓ Both sessions end their transactions and a subsequent update succeeds.

? Does killing the waiting session remove the head blocker?

? Does retrying immediately forever solve deadlocks safely?

Debrief and deliverable

A lock diagram, deadlock interpretation and verified cleanup.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Replication delay and reporting freshness

A reporting screen shows stale points while the primary accepts transactions. This is a supplied evidence exercise, not a configured replica.

- Application routing and replica freshness must be investigated together.
- Replication threads, errors and delay metrics need timestamps and context.
- An empty replica-status result on a standalone server is not a replication failure.

Your investigation sequence

1. Read the fixture and record provenance
2. Build a freshness table
3. Choose the next investigation
4. Transfer the method without copying

Read the fixture and record provenance

```
cd "$COURSE_ROOT"
cat shared/fixtures/replication.json
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Student recognizes NULL delay as unknown rather than healthy.
- ✓ Recommendation addresses stale reads and the replication error separately.

? Does a running receiver thread prove the applier is current?

? Can reporting freshness be inferred from one lag number alone?

Debrief and deliverable

A replication escalation note with evidence and routing tradeoffs.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

SQL Server requests, sessions and wait deltas

SQL Server is blamed for high CPU. Capture active work and distinguish current activity from cumulative counters.

- SQL Server 2022 diagnostic permissions differ from older examples; validate visibility first.
- Requests and sessions are related but not interchangeable.
- Wait statistics are cumulative; compare two captures over a known interval.

Your investigation sequence

1. Set connection details in the client shell
2. Confirm identity and database access
3. Capture baseline diagnostics
4. Observe a bounded workload
5. Compute interval wait deltas
6. Transfer the method without copying

Set connection details in the client shell

```
read -r -p 'SQL Server hostname or private IP: ' SQL_HOST
read -r -p 'Assigned SQL login: ' SQL_LOGIN
read -r -s -p 'SQL password: ' SQLCMDPASSWORD
printf '\n'
export SQL_HOST SQL_LOGIN SQLCMDPASSWORD
SQL_TLS_ARGS=()
command -v sqlcmd
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Report includes time window, permissions and correct units.
- ✓ Student separates historical high waits from interval changes.

? Can a sleeping session be the head blocker?

? Does highest total_worker_time identify the currently slowest query?

Debrief and deliverable

A SQL Server baseline and a wait-delta interpretation.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Blocking chains, Query Store and job failures

A deployment coincides with latency and a failed reporting job. Establish whether the symptoms share a cause.

- Identify the head blocker through sessions as well as active requests.
- Query Store compares plans and runtime intervals; correlation with deployment is not proof of causation.
- SQL Agent job outcome and step outcome need separate interpretation.

Observe before releasing the blocker



Sleeping sessions may own open transactions. Preserve the blocking relation before cleanup.

Open three SQL client sessions

```
sqlcmd "${SQL_TLS_ARGS[@]}" -S "tcp:$SQL_HOST,1433" -U "$SQL_LOGIN" -d SupportLab
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Capture a blocking relation and roll back both sessions.

✓ Compare Query Store duration/reads and identify the failed job step using live or explicitly synthetic evidence.

? Does a changed plan guarantee a regression?

? Is plan forcing always the safest first action?

? Can a failed job explain stale reports without explaining API latency?

Debrief and deliverable

A blocking worksheet, plan comparison and job-failure escalation.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Application/database correlation

An HTTP request took five seconds, but its database query took 40 milliseconds. Determine where the rest of the time went.

- Application latency includes pool waits, SQL time, retries and downstream work.
- Correlate IDs and UTC windows; acknowledge clock skew and missing instrumentation.
- Request IDs in app logs do not automatically appear in database logs.

Your investigation sequence

1. Read and reconcile the synthetic request timing
2. Measure the healthy live baseline
3. Introduce application-side delay
4. Compare latency with application evidence
5. Reset and confirm the comparison
6. Transfer the method without copying

Read and reconcile the synthetic request timing

```
cd "$COURSE_ROOT"
cat shared/fixtures/correlation.csv
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Timeline accounts for measured time without inventing an exact database-request-ID join.

✓ Learner distinguishes the sample app’s behavior from the supplied pooled-app evidence.

? Does slow HTTP prove slow SQL?

? Can retries amplify a partial outage?

Debrief and deliverable

A correlated timeline and a targeted escalation request.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Final database incident challenge

A reporting request regressed after a release while ordinary player lookup remains responsive. Distinguish competing database causes using a separate synthetic incident pack.

- Quantify changes in workload, query cost and result equivalence.
- Test competing causes with discriminating evidence.
- Define a reversible correction and bounded database-health checks.

Your investigation sequence

1. Prepare your submission workspace
2. Prepare database-specific evidence
3. Receive the incident and assign roles
4. Investigate within the time budget
5. Collect and inspect evidence
6. Propose one mitigation and define recovery

Prepare your submission workspace

```
cp -i "$COURSE_ROOT/shared/templates/incident-report.md"
"$HOME/support-lab/evidence/m2-09/incident.md"
date -u +%FT%TZ
nano "$HOME/support-lab/evidence/m2-09/incident.md"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Quantify the report regression and test at least two explanations.
- ✓ Propose a scoped change and compare instructor-released before/after evidence.
- ✓ Submit a bounded database-health automation design and an individual explanation.

? Do three unblocked snapshots rule out all intermittent blocking?

? Does a plan change alone prove which repair is correct?

Debrief and deliverable

A case report with calculations, alternative hypotheses, before/after comparison and a database-health design.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Reference shelf and completion checks

Use shared/SOURCES.md for official technical references.

Submit scripts, evidence, interpretations and recovery checks.

Keep private keys and passwords out of submissions.

Complete the core labs before attempting extension challenges.

Evidence 25 • Diagnosis 25 • Recovery 20 • Automation 20 • Communication 10